

后量子密码机原型（ZU3EG / AXU3EGB）说明文档

交付日期：2026-08-13 器件：xazu3eg-sfvc784-1-i（XCZU3EG, 70560 LUT / 360 DSP / 216 BRAM36） 代码分支：zu3eg-fpga-crypto 本文所有数字与日志均来自真实硬件运行，不是仿真、不是估算。

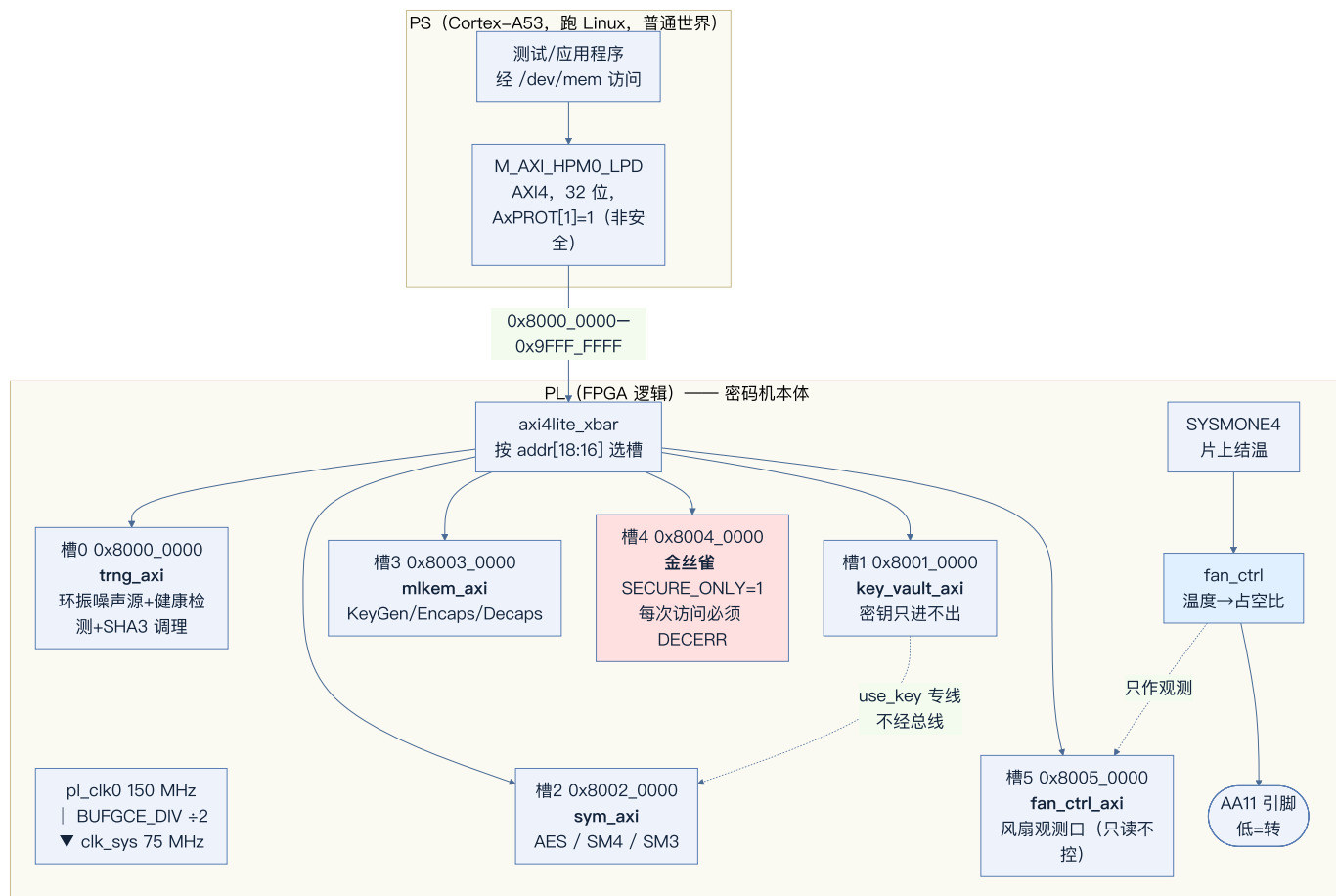
0. 这份文档能回答什么

- 做出来的是什么：**一台把 ML-KEM、AES/SM4/SM3、真随机数源、密钥仓与访问边界 全部放进 FPGA 逻辑（PL）的密码机原型，Linux（PS）只作为发命令的一侧。
- 它被验证到什么程度：**ML-KEM 三套参数集对 NIST 官方向量逐字节一致、对称与国密核对标准向量逐字节一致、密钥”能用但读不出来”有反证、真随机数源的实测最小熵、以及每一项的原始日志。
- 还有什么没做、为什么：**第 5 节逐条列出，每条注明卡在哪里。

不回答的：性能调优（原型不追性能）、功耗/电磁侧信道（明确不做）。

1. 项目架构

1.1 总体分工



后量子密码机原型总体架构

1.2 地址映射

基址 `0x8000_0000`，槽号取 `addr[18:16]`，每槽 64 KB。

地址映射是一一对应的：一个寄存器有且只有一个地址能访问到它。`axi4lite_xbar` 做完整译码——aperture 高位、槽号、槽内偏移的高位、32 位对齐，四条全部成立才转发，任何一条不成立就就地 DECERR，下游一个字节都收不到。

第一版不是这样：它只用 `addr[18:16]` 选槽、只把 `addr[7:0]` 交给从机，中间的 `addr[15:8]` 与 aperture 以上的高位整个被丢掉，于是每个寄存器都有大量镜像（`0x8001_0110` 与 `0x8001_0010` 是同一个寄存器）。

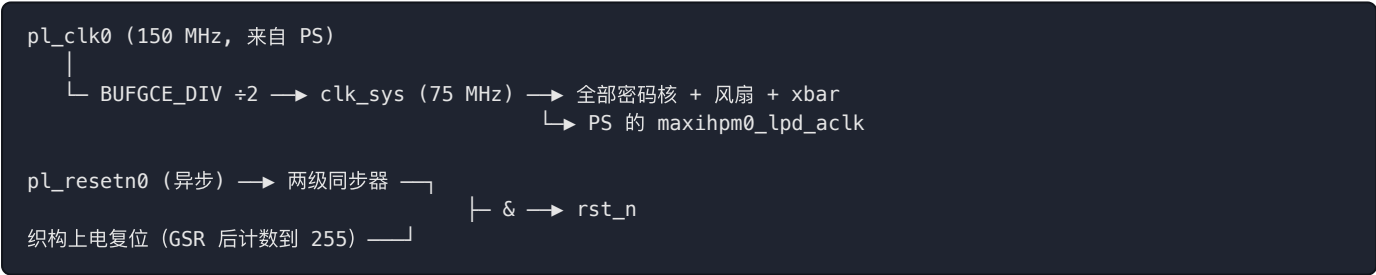
这一条为什么定为 P0，后来查 UG1085 才看清楚全部分量（见 5.5）：PS 侧根本没有任何保护单元覆盖这段 PL 窗口——XPPU 的 aperture 表里没有它，FPD_XMPU 也不在 `M_AXI_HPM0_LPD` 这条路径上。也就是说 PL 内的这道译码与防火墙是**唯一**一层，没有谁在后面兜底；镜像地址等于在唯一的防线上开了成千上万个等价入口，而且每一个都返回 OKAY、不留痕迹。

这一条由 `test_xbar` 判，判断是“从机有没有被碰到”而不是“回了什么响应码”：镜像地址在旧译码下**也会**回 OKAY，因为它确实读到了那个寄存器。其中一条用例把整个 64 KB 槽按字扫一遍，实测可达地址恰好是 `0x00` — `0xFC` 那 64 个，其余 16320 个全部 DECERR；板上复验 13/13。

地址	从机	SECURE_ONLY	说明
0x8000_0000	trng_axi	0	真随机数源。0x08 RDATA 每读一次弹一个 32 位字
0x8001_0000	key_vault_axi	0	密钥仓。密钥只能写入、只能经专线给对称核用，总线上读不到
0x8002_0000	sym_axi	0	AES-128/256、SM4、SM3
0x8003_0000	mlkem_axi	0	ML-KEM 512/768/1024, MODE[3:2] 选参数集
0x8004_0000	金丝雀 (key_vault_axi)	1	与槽 1 同一个模块，只差这个参数。存在的唯一目的是被拒绝
0x8005_0000	fan_ctrl_axi	0	风扇温度/占空比观测；拔掉它风扇照转

为什么功能从机是 **SECURE_ONLY=0**：板上 Linux 跑在普通世界，它经 `/dev/mem` 发出的每一笔事务 **AxPROT[1]** 都是 1。四个功能从机若都设成 1，Linux 一个寄存器也读不到，这一版就只能证明“全都拒绝”，证明不了算法对不对。所以功能核放开、另设一个**只差这一个参数**的金丝雀实例——它每次被拒，就是 AxPROT 门控在真硬件、真非安全 master 下确实生效的证据。生产形态下四个从机全为 1，由安全世界驱动（见第 5 节）。

1.3 时钟与复位



- 为什么分频到 75 MHz：最慢的核 **mlkem_decaps** 单独综合只收敛到 108.5 MHz，直接用 150 MHz 必然过不了时序。75 MHz 对它留了 45% 余量，这个余量是留给“接上总线、加了时钟树之后关键路径必然变长”那部分的。
- 为什么用 **BUFGCE_DIV** 而不是 **MMCM**：MMCM 要等 lock、要排复位时序，多一组会出错的状态；**BUFGCE_DIV** 是纯分频，上电即有。
- 为什么还要织构自生的上电复位：配置完成后 FPGA 触发器一律为 0（GSR），所以那个计数器必然从 0 开始。有了它，即使 **p1_resetrn0** 因运行时重配没有正确释放，设计也能自己起来——少一个“只在真机上才暴露”的依赖。

2. 代码说明

2.1 目录导览

目录	模块数	职责
<code>hardware/rtl/mlkem/</code>	27	ML-KEM 全套：NTT、基乘、蒙哥马利/Barrett 约减、采样、压缩、编解码、KeyGen/Encaps/Decaps 三个整核
<code>hardware/rtl/mldsa/</code>	13	ML-DSA 的算子（NTT、舍入、提示位、采样）——已实现未整核化
<code>hardware/rtl/keccak/</code>	2	<code>keccak_f1600</code> 置换 + <code>sha3_core</code> 海绵（G/PRF/XOF/H 四种用法共用）
<code>hardware/rtl/sym/</code>	6	AES-128/256、SM4、SM3
<code>hardware/rtl/trng/</code>	6	环振噪声源、健康检测（RCT/APT）、SHA3 调理、FIFO、AXI 封装
<code>hardware/rtl/bus/</code>	8	AXI4-Lite 防火墙、密钥仓、各核的 AXI 封装
<code>hardware/rtl/board/</code>	2	交叉开关 <code>axi4lite_xbar</code> 、板级顶层 <code>zu3eg_hsm_top</code>
<code>fpga/fan_ctrl/</code>	4	风扇温控（与密码逻辑目录分离，两边只共用时钟和复位）
<code>hardware/tb/cocotb/</code>	—	197 个测试的对拍台
<code>hardware/tb/lint/</code>	—	厂商原语的空壳，只给 lint 用，不参与综合

2.2 顶层怎么集成

`hardware/rtl/board/zu3eg_hsm_top.v` 一个文件把所有东西接起来：

- 例化 PS IP（`zynq_ultra_ps_e_0`），取 `pl_clk0` / `pl_resetn0` / `M_AXI_HPM0_LPD`；
- `BUFGCE_DIV` 分频出 `clk_sys`，必须写在 PS 例化之前（见 2.3 第一条）；
- AXI4 的 `bid` / `rid` / `rlast` 由 PL 回驱（见 2.3 第二条）；
- `axi4lite_xbar` 按 `addr[18:16]` 分发到 6 个槽；
- `fan_sysmon` + `fan_ctrl` 直连 `AA11`，与 AXI 无关。

2.3 重要设计决策（每条都有代价换来的理由）

① `M_AXI_HPM0_LPD` 是 AXI4，不是 AXI4-Lite。第一版把它当 Lite 接，`bid` / `rid` / `rlast` 三个”由 PL 驱动、送回 PS”的信号全部悬空。PS 在等 `rlast` 结束读突发，等不到就永远不返回，CPU 卡死在那一条读指令上。综合、布线、时序、bitstream 全部正常，载进板子也显示 operating——这个 bug 只在真机上现形。AXI4-

Lite 本质是”突发长度恒为 1 的 AXI4”，所以 `rlast = rvalid`，`bid / rid` 回显 `awid / arid`。实现流程里为此加了一条综合后断言（见 4.5）。

② 顶层里”先声明后使用”不是风格问题。若引用了后面才声明的 `wire`，Vivado 不报错，而是给那个端口新建一条同名的 无驱动网络。这让板子挂死两次、断电两次。Icarus 对同样写法直接报 `Unable to bind wire`——所以 lint 里专门给厂商原语写了空壳，好让板级顶层 照样进 lint（把它排除出去等于关掉唯一一道自动门）。

③ `SECURE_ONLY` + AxCROT 门控。`axi4lite_firewall` 在 `SECURE_ONLY=1` 时要求 `AxCROT[1]==0`，否则回 DECERR。被拒的读绝不弹出 FIFO——否则一个拿不到数据的非安全读也能把随机字冲掉，等于给普通世界一个耗尽熵池的手段。

④ 密钥仓：密钥只进不出。密钥经总线写入，经专线 `use_key` 直接进对称核，总线上没有任何读回路径。第 4.1 节的边界反证扫了两个从机各 256 字节，逐字比对密钥的每一个字。

⑤ 常数时间比较（ML-KEM Decaps 隐式拒绝）。密文比对逐字节全比完再选，绝不”一发现不同就早退”。第 4.4 节在真硅上量了 两条路径的耗时分布。

⑥ 风扇温度必须从 PL 自己读，不能让 Linux 读了写寄存器。风扇是散热不是外设：它得在没有 Linux 的时候也工作——开机那几十秒、U-Boot 里停着的时候、内核挂死的时候。所以结温直接从 PL 的 SYSMONE4 经 DRP 读，整条链路只依赖 PL 时钟；AXI 那一路只是观测口。

⑦ 风扇的四道安全，方向永远是「多吹」。

#	条件	动作
①	最低档	25% 而不是 0——把风扇停死不是省电是赌运气
②	≥80°C	强制 100%，降到 74°C 才解除（进得早、出得晚）
③	长时间拿不到有效温度	强制 100%——温度未知时唯一安全的假设是”可能很热”
④	读数长时间一个比特不变	强制 100%——见下

第 ④ 条是上板之后补的。第一版 SYSMON 配置写错，DRP 照常应答、寄存器里 照样有一个看着很合理的 32.5°C，只是 ADC 根本没在转换。“读不到”永远不成立，第 ③ 条完全挡不住。真实结温永远在抖，所以”完全不变”本身就是故障信号。

3. 测试结果 · 功能

3.1 ML-KEM 三套参数集（NIST ACVP 官方向量，20/20）

向量来源：`vectors/acvp/ML-KEM-{keyGen,encapDecap}-FIPS203/`（NIST ACVP-Server 的 `prompt.json` + `expectedResults.json` 配对）。参数集只靠 `MODE` 寄存器的 `pset` 字段切换，长度由 RTL 自己按 `pset` 算，软件不报长度，也就报不错。

ML-KEM 从机 VERSION = 0x00010000

[① ML-KEM 三套参数集 · NIST ACVP 官方向量]

```
ok ML-KEM-512 KeyGen tc1: ek 800 + dk 1632 字节逐字节一致
ok ML-KEM-512 KeyGen tc2: ek 800 + dk 1632 字节逐字节一致
ok ML-KEM-768 KeyGen tc26: ek 1184 + dk 2400 字节逐字节一致
ok ML-KEM-768 KeyGen tc27: ek 1184 + dk 2400 字节逐字节一致
ok ML-KEM-1024 KeyGen tc51: ek 1568 + dk 3168 字节逐字节一致
ok ML-KEM-1024 KeyGen tc52: ek 1568 + dk 3168 字节逐字节一致
ok ML-KEM-512 Encaps tc1: K 32 + c 768 字节逐字节一致
ok ML-KEM-512 Encaps tc2: K 32 + c 768 字节逐字节一致
ok ML-KEM-768 Encaps tc26: K 32 + c 1088 字节逐字节一致
ok ML-KEM-768 Encaps tc27: K 32 + c 1088 字节逐字节一致
ok ML-KEM-1024 Encaps tc51: K 32 + c 1568 字节逐字节一致
ok ML-KEM-1024 Encaps tc52: K 32 + c 1568 字节逐字节一致
ok ML-KEM-512 Decaps tc76: K 32 字节逐字节一致
ok ML-KEM-512 Decaps tc77: K 32 字节逐字节一致
ok ML-KEM-768 Decaps tc86: K 32 字节逐字节一致
ok ML-KEM-768 Decaps tc87: K 32 字节逐字节一致
ok ML-KEM-1024 Decaps tc96: K 32 字节逐字节一致
ok ML-KEM-1024 Decaps tc97: K 32 字节逐字节一致
```

3.2 对称与国密 + 边界反证 + AxPROT 对照 + TRNG 健康检测 (24/24)

以下是 `board/RESULT_hwtest.txt` 的完整原文：

```
PL 密码机硬件自测 @ 0x80000000
mlkem_axi VERSION = 0x00010000
```

[ML-KEM-512 NIST ACVP 向量]

```
PASS KeyGen tc0: ek 800 + dk 1632 字节逐字节一致
PASS KeyGen tc1: ek 800 + dk 1632 字节逐字节一致
PASS KeyGen tc2: ek 800 + dk 1632 字节逐字节一致
PASS Encaps tc0: K 32 + c 768 字节逐字节一致
PASS Encaps tc1: K 32 + c 768 字节逐字节一致
PASS Encaps tc2: K 32 + c 768 字节逐字节一致
PASS Decaps tc0: K 32 字节逐字节一致
PASS Decaps tc1: K 32 字节逐字节一致
PASS Decaps tc2: K 32 字节逐字节一致
```

[对称与国密 FIPS 197 / GB-T 32907 / GB-T 32905]

```
PASS 密钥仓: 三把密钥装载并 COMMIT 成功
PASS AES-128 加密: FIPS 197 C.1 逐字节一致
PASS AES-128 解密: 回到原文
PASS AES-256 加密: FIPS 197 C.3 逐字节一致
PASS SM4 加密: GB/T 32907 A.1 逐字节一致
PASS SM4 解密: 回到原文
PASS SM3("abc"): GB/T 32905 A.1 逐字节一致
```

[密码边界: 密钥能用, 但读不出来]

```
PASS 扫完两个从机各 256 字节 (48 个读得到、80 个被防火墙拒): 密钥的 4 个字一个都没出现, 而密文正确
```

[AxPROT 门控: 非安全 master 访问 SECURE_ONLY=1 的实例]

```
PASS 金丝雀读被总线拒绝 (DECERR/SIGBUS) — AxPROT 门控在真硬件上生效
PASS 对照: SECURE_ONLY=0 的同一模块读回 VERSION=0x00010000
```

[TRNG]

```
PASS 启动健康检测通过 (SP 800-90B §4.3, 1023 个样本)
PASS 无 RCT / APT 告警
PASS 取到 256 个随机字, 交付计数 256
PASS 一比特占比 0.5035 (4125/8192), 无明显偏置
PASS 256 个字里没有相邻重复
注: 以上**不是**最小熵评估。真实最小熵要导出原始比特
跑 SP 800-90B 的 EntropyAssessment, 见 ring_osc.v 的说明。
```

```
=====
通过 24, 失败 0
```

边界反证怎么读: 扫描两个从机各 256 字节, **48 个读得到、80 个被防火墙拒** ——被拒的那 80 个在软件侧表现为 SIGBUS (工具带全局 SIGBUS 处理继续扫)。关键结论是那 48 个读得到的字里, **密钥的 4 个字一个都没出现**, 而同一把密钥算出的密文是正确的。这两条合起来才叫“能用但读不出来”, 缺任何一条都不成立。

3.3 TRNG 实测最小熵

从板上导出 **1,048,576 个调理前原始噪声比特** (**RAW_TAP=1** 的表征版 bitstream; 产品版里这条通路连寄存器一起不存在, 不是“读了返回 0”)。

必须用调理前的: 调理器 (SHA-3 海绵) 的输出无论输入熵多低看着都像均匀 随机, 拿 **RDATA** 跑评估会得到一个非常漂亮、但毫无意义的数字。

```
$ python3 tools/sp800_90b.py /tmp/trng_bits.bin
样本数 1048576, 字母表大小 2
MCV (最常见值) 0.996191
Collision (碰撞) 0.871234
Markov (马尔可夫) 0.996108
Compression (压缩) 1.000000
t-Tuple 0.923437
LRS (最长重复子串) 0.993807
MultiMCW (窗口最常见) 0.999567
Lag (滞后) 0.987625
MultiMMC 0.994768
LZ78Y 0.993032
**最小熵 (取最小者)** 0.871234
```

H = 0.871234 比特/样本 (十项取最小, 由碰撞估计器卡住)。

原始数据本身: 1 的比例 0.500064、最长游程 19 (1M 比特的期望约 20)、lag 1–8 自相关全部 < 0.4%、32 位
字内各位置无结构性偏置; 采集期间健康检测 无报警 (alarm=0 rct=0 apt=0)。

⚠ 工具来源要说准: 构建机没有 NIST 官方 EntropyAssessment, 也没有外网。tools/sp800_90b.py 是按 SP 800–90B (2018–01 最终版) §6.3.1–6.3.10 逐条 实现的, 不是官方工具的输出。可信度靠规范每一节自带的算例逐字复现:

```
$ python3 tools/sp800_90b.py --selftest
6.3.1 MCV 期望 0.5363 实得 0.5363 ✓
6.3.2 碰撞 期望 0.4483 实得 0.4483 ✓
6.3.3 马尔可夫 期望 0.7610 实得 0.7606 ✓
6.3.5 t-Tuple(cut=3) 期望 0.2730 实得 0.2731 ✓
6.3.6 LRS(cut=3) 期望 0.6146 实得 0.6146 ✓
6.3.8 Lag(D=3) 期望 0.7350 实得 0.7349 ✓
6.3.9 MultiMMC(D=3) 期望 0.0755 实得 0.0755 ✓
6.3.4 压缩(d=4) 期望 0.1345 实得 0.1372 ✓
6.3.7 MultiMCW(w=3,5,7,9) 期望 0.3908 实得 0.3909 ✓

自检: 全部通过
```

(第十项 LZ78Y 规范未给算例, 它与其他四个预测器共用同一段已验证的 P'_global / P_local 收尾计算。)
这个实测值推翻了两个既有参数。旧的 RCT_CUTOFF=41 / APT_CUTOFF=793 是按 H=0.5 假设算的, 拿实测 H 重算, 两个都站不住, 而且坏的方向不一样:

参数	旧值在实测熵下的真实含义	后果
RCT=41	相当于 $\alpha=2^{-34.8}$; 样本率约 9.4 M/s	平均 55 分钟误报一次, 对常开熵源太频繁
APT=793	触发概率 5.5×10^{-52}	这道检测从来不会触发, 等于不存在; 过去所有“APT 没报警”的记录, 一条证据都不构成

按 $\alpha=2^{-40}$ 重算 (约 33 小时一次虚警, 对这个样本率是合理预算): RCT 41→47, APT 793→672。 α 不照抄规范里那个 2^{-20} ——在 9.4 M/s 下它是 每 0.11 秒误报一次, 没法用; α 要按样本率选。

公式先在 H=0.5 下重算, 得到的正是 RTL 里原有的 41/793——先证明公式与当初 的出处对得上, 再拿它往前推。

⚠ 采集是**有间隙**的（抽头 FIFO 64 字深，满了丢新的，**绝不回压噪声源**——回压会改变被评估的那条流本身）。这对最小熵估计没有影响，但 **重启测试（restart tests）** 不能用这份数据。

3.4 风扇温控（真硅上的完整闭环）

表征版 bitstream（阈值压到 30/32/34/36°C，因为这块板在产品阈值 45°C 下 **热不起来**——风扇完全停转 + 四核满载 + PL 密码核循环，180 秒结温只从 34.1 升到 36.5°C 就趋平）。控制律是同一份 RTL，只有常数不同。

```
(节选自 charlog.txt, C10 = 结温×10)
base    duty=25 step=0 C10=294      ← 29.4°C, 0 档 25%
heat    duty=0  step=1 C10=296      ← 覆盖压住占空比, 档位照常跟温度走
heat    duty=0  step=1 C10=313
heat    duty=0  step=2 C10=319      ← 跨过 32°C
heat    duty=0  step=2 C10=334
heat    duty=0  step=3 C10=337      ← 跨过 34°C
release duty=60 step=2 C10=331      ← 放开覆盖, 占空比立刻对上当前档位
release duty=60 step=2 C10=317
cool    duty=40 step=1 C10=311      ← 降温, 逐级退回
cool    duty=40 step=1 C10=302
```

验的性质	结果
传感器是活的	我的 DRP 读数 vs PS 侧 AMS（ 完全不同的通路 ）5 分钟内吻合到 1°C 内；ADC 的最高/最低温寄存器已离开复位值（ 0x20 = 0xA41C、 0x24 = 0xA17A，非 0x0000/0xFFFF）
上升沿	29.4→33.7°C，档位 0→1→2→3 逐级跨过三个阈值
下降沿 + 迟滞	43.6°C 时在 1 档 40%，掉到 T1_DN(40°C) 以下才回 0 档 25%
覆盖优先级	放开覆盖后占空比立刻对上当前档位（2 档→60%）
散热代价	25% 占空比 ≈ 原厂满速（28.5 vs 29.8°C）——安静不是拿温度换的

3.6 边界完整闭合：安全世界能驱动，普通世界不能

3.2 只证明了一半——**SECURE_ONLY=1** 的金丝雀**拒绝普通世界**。另一半“安全世界能驱动”没法从普通世界证：Linux 跑在 EL1-NS，它发出的每一笔事务 **AXPR0T[1]** 恒为 1，**根本发不出安全事务**。

做法：给 BL31（跑在 EL3、**SCR_EL3.NS=0**）加一个最小的 SiP 调用（**0x8200ff10**），在 EL3 上读一个 PL 地址，值从 SMC 返回。Linux 侧用一个内核模块（**board/kmod/pmsec.c**）发这个 SMC——普通世界的用户态发不出 SMC，而 5.4 的 **zynqmp_eemi_ops** 没导出 mmio 读写。

结果（同一块板、同一份 PL bitstream、同一次运行）：

```
=== 从哪个槽启动的 ===
pmsec: CSU_MULTI_BOOT @0xffca0010 ret=0 val=0x00000004

=== ① 对照: EL3 读 SECURE_ONLY=0 的 ML-KEM (应当成功) ===
pmsec: EL3 安全读 0x80030000 → ret=0 val=0x00010000

=== ② 决定性一步: EL3 读 SECURE_ONLY=1 的金丝雀 ===
pmsec: EL3 安全读 0x80040000 → ret=0 val=0x00010000

=== ③ 同一地址, 普通世界 (用户态 /dev/mem, AXPROT[1]=1) ===
读 0x80030000 ...
0x80030000 = 0x00010000
↑ SECURE_ONLY=0 的核, 普通世界读得到
读 0x80040000 ...
pay_sec.sh: line 38: 826 Bus error   rdtreg 0x80040000
↑ 金丝雀, 普通世界读的结果 (rc=135: 被拒时进程收 SIGBUS)
```

四格真值表齐了：

	0x80040000 金丝雀 (SECURE_ONLY=1)	0x80030000 对照 (SECURE_ONLY=0)
安全世界 EL3 (AXPROT[1]=0)	读到 0x00010000 ✓	读到 0x00010000
普通世界 EL1-NS (AXPROT[1]=1)	SIGBUS / DECERR ✓	读到 0x00010000

右边那一列是关键的对照：同一个普通世界能读到 SECURE_ONLY=0 的核，所以左上/左下的差别确实来自 AXPROT 门控，而不是”这个地址本来就不可达”。

途中排除的两条更轻的路（否定结果本身有信息量）

- ATF 现成的 SiP：没有任何”EL3 自己做 mmio”的调用。
- PM_MMIO_READ：它是把请求转给 PMU 执行的。板上实测三个 PL 地址 全返回 2002 (XST_PM_NO_ACCESS)，而且对 SECURE_ONLY=1 的金丝雀 和 =0 的核一视同仁。也就是说拒的是 PMUFW 的地址白名单、不是 AXPROT 门控 —— 这条路即便”成功”也证明不了门控，何况它没成功。这一点必须写清楚：很容易被误读成”安全主控读不到，所以门控生效”，那是错误的推论。

没有 JTAG 怎么敢刷

镜像写进 BOOT0004.BIN 这个新槽（不覆盖任何已有槽），靠 CSU_MULTI_BOOT 指过去。而 multiboot 由 POR 清零 —— 这份镜像只在热重启 链内有效，任何一次断电都自动回到 BOOT.BIN（黄金）。这是没有 JTAG 时 能拿到的最好兜底：最坏情况是”要断一次电”，而不是”砖”。

BL31 的编译参数与已知 10/10 能启动的 atf_build_v50 逐字相同（SPD=opteed DEBUG=1 LOG_LEVEL=50），镜像结构照 boot_fix_quiet.bif —— 变量只有那一个 SiP 分支。核对过装载布局与旧版一致，并用反汇编确认 0x8200ff10 的比较真的在二进制里（旧版一条都没有；”编过了”和”生效了”是两回事）。golden BOOT.BIN / BOOT0001 / BOOT0002 / BOOT0003 全程 md5 未变。

第一次没成功，以及为什么

第一版只加了 SiP 分支，SMC 发出去再也不返回。原因是 BL31 的页表里 根本没有 PL 那段地址：bl_regions[] 只映射 BL31 自己那几段，plat_arm_get_mmap() 给的是 0xFF00_0000 那一带的外设。发

SMC 的那个核在 EL3 取数时翻译错误、卡死在异常处理里，而**其余三个核照常跑 Linux** —— 所以板子看起来还活着，最后是 `plharness.sh` 那条 480 秒看门狗把它重启的。

这个”半死不活”的形状很容易被误读成 SMC ABI 写错了。补上 2 MB 的设备映射（覆盖全部六个槽）之后一次通过。同时把 SiP 的地址检查**收到与映射完全一致** —— 检查放得比映射宽，就等于放行一个会在 EL3 翻译错误的地址，那是把”读寄存器的口子”做成了”挂板子的口子”。

3.5 PL 加载与网卡（dmesg 原文）

```
[15451.992467] xilinx_axienet 80000000.ethernet eth0: Link is Up - 1Gbps/Full
[15612.678785] fpga_manager fpga0: writing zu3eg_hsm_fan.bit to Xilinx ZynqMP FPGA Manager
[15618.334326] fpga_manager fpga0: writing factory.bit to Xilinx ZynqMP FPGA Manager
[15622.596777] xilinx_axienet 80000000.ethernet: TX_CSUM 0
[15622.607485] libphy: Xilinx Axi Ethernet MDIO: probed
[15625.699171] xilinx_axienet 80000000.ethernet eth0: Link is Up - 1Gbps/Full
[15674.935047] fpga_manager fpga0: writing fanquiet.bit to Xilinx ZynqMP FPGA Manager
[15682.223443] xilinx_axienet 80000000.ethernet eth0: Link is Up - 1Gbps/Full
```

eth0 就在 PL 里（`80000000.ethernet` 是厂家设计里的 AXI Ethernet）。所以每次重配 PL 都要先解绑 PL 驱动、装完再绑回来——带着活的 AXI 主口 **重配 fabric** 会把总线挂死，而 AXI 一挂连 `sysrq` 都写不进去。这条纪律固化在 `board/plharness.sh` 里，上层只写 payload。

4. 测试结果 · 性能与实现

4.1 ML-KEM 吞吐（真硅，@75 MHz）

每参数集每操作 20 次平均：

参数集	KeyGen	Encaps	Decaps
ML-KEM-512	1.08 ms (924 次/秒)	0.75 ms (1339 次/秒)	0.98 ms (1018 次/秒)
ML-KEM-768	1.65 ms (605 次/秒)	1.12 ms (895 次/秒)	1.44 ms (694 次/秒)
ML-KEM-1024	2.27 ms (440 次/秒)	1.57 ms (636 次/秒)	1.96 ms (510 次/秒)

⚠ **含软件逐字节搬进搬出的开销**（AXI 每字节一次读写），不是纯硬件算核 时间。原型验证不追性能，这组数只作横向对照：三套参数集的比例约 **1 : 1.5 : 2.1**，与 $k=2/3/4$ 的理论工作量相符——这一点比绝对值有意义。

4.2 常数时间抽测（Decaps 隐式拒绝路径）

合法密文与改了一个比特的密文各跑 200 次：

```
[③ Decaps 常数时间抽测（合法密文 vs 隐式拒绝）]
合法密文   中位 1.4405 ms  最小 1.4404  最大 1.5258
隐式拒绝   中位 1.4405 ms  最小 1.4404  最大 1.4511
中位数相对差 0.000%（各 200 次）
ok         两条路径耗时无法区分（相对差 0.000% < 0.5%）——密文比对确实是全比完再选，没有早退
```

中位数相对差 0.000%。真早退的话差异会是数量级的，而不是百分之几。

测的是软件侧能观测到的那一层延迟，也正是计时攻击真正能拿到的那一层。它不覆盖功耗/电磁侧信道（本次不做，也不假装做了）。

4.3 资源与时序（完整实现，含布局布线）

CLB LUTs	35592	0	70560	50.44
CLB Registers	25916	0	141120	18.36
CARRY8	204	0	8820	2.31
Block RAM Tile	15.5	0	216	7.18
DSPs	140	0	360	38.89
Bonded IOB	1	1	252	0.40

WNS(ns)	TNS(ns)	TNS Failing Endpoints	TNS Total Endpoints	WHS(ns)	THS(ns)	THS Failing
3.174	0.000	0	79055	0.001	0.000	0
All user specified timing constraints are met.						

项	值
LUT	35611 / 70560 (50.47%)
寄存器	25916 (18.36%)
BRAM	15.5 / 216 (7.18%)
DSP	140 / 360 (38.89%)
外部管脚	1 (风扇 AA11)
WNS / WHS	+3.643 ns / +0.013 ns @75 MHz
对应 Fmax	约 99 MHz (周期 13.333 ns — WNS 3.174 ns = 10.159 ns 关键路径)

TRNG + 密钥仓 + AES/SM4/SM3 + ML-KEM 三核 + 金丝雀 + 译码 + 风扇温控 + PS 接口，占这颗片子刚好一半。

说明：上表是产品版实现的实测值，bitstream md5 `d689d753276989607ce34c52c8e3eb3b`。

与审计前那一版的差：+419 LUT (49.85% → 50.44%)、+92 寄存器，BRAM 与 DSP 不变，WNS 从 +3.469 降到 +3.174 ns。多出来的是完整地址译码、ML-KEM 的 BRAM 擦除机与参数校验、TRNG 的取样 FIFO (见 4.6)。

⚠ 保持时间余量值得盯一下：WHS 从 +0.011 → +0.013 → **+0.001 ns**。仍然是正的，Vivado 的 sign-off 判它通过，实现流程里那条“WHS < 0 不出 bitstream”的断言也放行了。但 1 ps 基本等于零，而**保持时间违例不是降频能解决的**——建立时间余量再大也救不了它。这一版板上跑通了 (见 4.7)，但下一次动 RTL 时这个数要重新看，不能默认它还在正的一侧。

关于可复现性，先前一版文档写过“同一份 RTL 两次实现有 35104→35173 LUT 的波动”——**那句话是错的，已删**：那两次跑的其实是不同的 RTL (一次在 SYSMON 修复之前)。后来拿**确实相同**的输入验过：lint 清理 (位宽修正 + 删死代码) 前后各实现一次，资源与时序**逐位相同** (当时是 35173 LUT / WNS +3.469 / WHS +0.011)。也就是说这条流程在这台构建机上是确定性的，而那些 lint 改动是综合中性的 (Vivado 本来就把死代码优化掉了，位宽修正不改变实现出来的逻辑)。

4.4 仿真回归 (197 项)

```
$ ./tools/rtl_sim.sh
✓ test_keccak          keccak_f1600          TESTS=5 PASS=5
✓ test_sha3_core       sha3_core          TESTS=9 PASS=9
✓ test_xbar            axi4lite_xbar       TESTS=6 PASS=6
✓ test_firewall        axi4lite_firewall   TESTS=6 PASS=6
✓ test_axi             pqc_accel_axi       TESTS=16 PASS=16
✓ test_key_vault_core  key_vault          TESTS=4 PASS=4
✓ test_key_vault       key_vault_axi       TESTS=6 PASS=6
✓ test_mlkem_axi       mlkem_axi           TESTS=9 PASS=9
✓ test_aes             aes_core            TESTS=5 PASS=5
✓ test_sm4             sm4_core            TESTS=6 PASS=6
✓ test_sm3             sm3_core            TESTS=6 PASS=6
✓ test_sym_vault       sym_vault_top        TESTS=5 PASS=5
✓ test_fan_ctrl        fan_ctrl            TESTS=7 PASS=7
✓ test_sysmon_drp      sysmon_drp          TESTS=3 PASS=3
✓ test_trng_health     tb_trng_health       TESTS=8 PASS=8
✓ test_trng_source     trng_source          TESTS=3 PASS=3
✓ test_trng_cond       trng_cond            TESTS=4 PASS=4
✓ test_trng_top        trng_top             TESTS=4 PASS=4
✓ test_trng_nodrop     trng_top             TESTS=2 PASS=2
✓ test_trng_drops      trng_top FIFO_DEPTH=2 TESTS=1 PASS=1
✓ test_trng_axi        trng_axi             TESTS=6 PASS=6
✓ test_trng_top_alarm  trng_top RCT_CUTOFF=2 TESTS=4 PASS=4
✓ test_trng_raw        trng_top RAW_TAP=1   TESTS=3 PASS=3
```

全部通过 (197 个测试)

Verilator lint 同步全过 (70 个模块, `./tools/rtl_lint.sh`)。

4.5 实现流程里的断言 (每条都是代价换来的)

`hardware/syn/impl_bitstream.tcl` 在综合后立刻检查，不合格就中止——避免“跑完三十多分钟才发现”或“载进板子才发现”：

```
断言通过: fan -> PACKAGE_PIN AA11 / LVCMOS33
断言通过: SYSMONE4 SIM_DEVICE = ZYNQ_ULTRASCAL
断言通过: maxihpm0_lpd_aclk <- u_div/0
断言通过: maxigp2_rlast <- u_xbar/s_rvalid_reg/Q u_xbar/maxigp2_rvalid
断言通过: maxigp2_bid <- awid_r_reg[15]/Q
```

断言	换来它的代价
PS 的 <code>aclk</code> / <code>rlast</code> / <code>bid</code> / <code>rid</code> 必须有驱动	两次板子挂死、两次断电
<code>fan</code> 必须落在 <code>AA11</code>	风扇错了没有任何运行时症状，只会安静地让芯片变热
<code>SYSMONE4</code> 的 <code>SIM_DEVICE</code>	一整轮三十多分钟的实现，最后一步 DRC 才失败
WNS/WHS < 0 不出 bitstream	时序不收敛的 bit 上板，出来的错像“算法写错了”，查起来极贵

4.6 一轮代码审计修掉的五个 RTL 硬伤

这五条是外部审计指出来的，五条都成立。共同点值得单独说一句：它们全都活在既有测试的盲区里——不是测试写得少，是每一条的判据都恰好落在测试没有在看的那个维度上。所以每修一条，都配一个能真正抓到它的确定性用例，并且在修复前的 RTL 上跑一遍确认它确实变红（下表最后一列）。不做这一步，“修好了”这句话就只是一个未经检验的断言。

#	问题	判据为什么以前抓不到	修法	用例	旧 RTL 上
①	<code>axi4lite_xbar</code> 地址别名：只用 <code>addr[18:16]</code> 选槽、只传 <code>addr[7:0]</code> ，其余地址位全丢	xbar 根本没进 <code>cocotb</code> ；而且镜像地址回的是 OKAY，只看响应码永远看不出异常	完整译码（aperture / 槽号 / 槽内偏移高位 / 对齐），任一条不成立就地 DECERR	<code>test_xbar</code> （6 条，含 64 KB 全槽扫描）	4/6 红

① 的分量后来还被放大了一层：查 UG1085 才确认 PS 侧没有任何保护单元覆盖这段 PL 窗口（5.5），所以这道译码是唯一一层强制点，镜像地址没有任何东西在后面兜底。| ② | `mlkem_axi` 的 `zeroize` 只清指针，两块 8 KB BRAM 里的旧 `dk` 原封不动 | 旧用例的判据是 `OUT_LEN==0`、`IN_PTR==0`——那只证明“软件读不到了”，正文还在不在它答不了 | 上升沿触发的擦除机：两块 BRAM 并行逐地址写 0（8192 拍），期间 `WIPING=1`、拒读拒写拒启动，写回 `SLVERR` 而不是静默丢弃 | `test_mlkem_axi` 两条，判据是读回 8192×2 个存储单元全为 0 | 4/8 红 || ③ | 防火墙与密钥仓的 `tamper` 只看锁存位，有一拍宽的 `TOCTOU` 窗口 | 既有用例都是“tamper 之后”，而问题恰恰在“tamper 那一拍” | 判据改成 `tamper \|\|` `tamper_latched`，组合项在前 | `test_firewall`（同拍/早一拍/正例三档）、`test_key_vault_core`（纯组合，不推时钟） | 3/6、1/4 红 || ④ | `TRNG` 调理器只在 `S_ABSORB` 收样，置换与挤出期间的样本被丢掉 | 没有任何用例去比“健康检测吃到的”与“调理器吃到的”是不是同一条流 | 源与调理器之间加 32 深取样 FIFO；溢出饱和计数暴露在 `0x34` | `test_trng_nodrop` 逐比特对比两条流；`test_trng_drops` 把 FIFO 压到 2 深反证计数是活的 | 红——首个分歧恰在第 1088 位，正好第一次置换处 || ⑤ | `ML-KEM` 的 `mode` / `pset` 各 2 位而只有 0/1/2 有意义，值 3 未被拒 | 没有 negative test | `START` 时判，非法则置 `PARAM_ERR` 且不启动任何核 | `test_mlkem_axi` 的 negative test（3 组组合） | 红 |

另有两处是上板才暴露的，仿真里全绿（详见 4.7）：调理器出口被输出 FIFO 卡住时仍会丢样；START 被拒之后上一次的 DONE / OUT_LEN 还留着。两处都已修， 并各补了一条把板上那个姿势搬进仿真的用例。

关于 ④ 需要多说一句，因为它最容易被当成小事。丢样的比例只有约 0.6%，“损失的是 熵率不是安全性”这句话就熵率而言并没有错。真正的问题在别处：SP 800-90B 的整套 论证建立在被评估的序列、被检测的序列、被使用的序列是同一个这条前提上。健康检测与 RAW_TAP 吃的是每一个样本，调理器吃的是其中一部分 —— 于是拿 A 序列 算出来的 $H = 0.871234$ 被用来给 B 序列记熵账。这是口径问题，不是精度问题， 所以不能靠”比例很小”糊过去。

关于 ② 也补一句：key_vault 用寄存器阵列所以擦除是一拍完成的（见该文件头）， BRAM 没有这个待遇 —— 一定存在”擦了一半”的窗口。设计上的选择是把这个窗口 明确暴露成一个状态位（STATUS[4] = WIPING）并在此期间关闭一切读写， 而不是假装它不存在。

新增的寄存器位（软件可见）：

位置	名称	含义
mlkem_axi STATUS[4]	WIPING	BRAM 擦除进行中。期间读 OUT_DATA 得 0、写一律回 SLVERR
mlkem_axi STATUS[5]	PARAM_ERR	上一次 START 因 mode / pset 非法被拒；下一次合法 START 清除
trng_axi 0x34	DROPS	取样 FIFO 溢出计数（饱和）。正常恒为 0；不为 0 即表示熵账口径已断

上板复验：哪几条能证，哪几条证不了

五项里能从板上的 Linux 观测到的只有三项半。把界限先划清楚，比事后解释更有用：

#	板上	为什么
① 地址别名	能证（除”未对齐”一档）	镜像地址必须 DECERR。未对齐那档证不了： /dev/mem 映出来是 Device 内存，未对齐的 32 位访问在 CPU 上就报对齐异常，总线根本没看见 —— SIGBUS 来自哪一层分不开
② BRAM 真擦除	半能证	WIPING 的行为与时长能证；“两块 BRAM 逐字节为 0”证不了，而这正是设计对的表现：软件本来就没有路径能读到那些字节
③ tamper 同拍	证不了	这一版 bitstream 里 tamper 恒接 0（ zu3eg_hsm_top.v ），板上没有东西能拉起它；即便能，“与总线握手同一拍”是拍级巧合，用户态安排不了
④ TRNG 不丢样	能证（必要条件）	读 0x34 必须为 0。它不等于”两条流逐比特相同”（那个由仿真判），但它防的是另一件事：真硅时序与仿真不同，忙的时间若比估计的长，FIFO 会真的溢

#	板上	为什么
⑤ 非法参数	能证	PARAM_ERR 置位、BUSY 不动、OUT_LEN 为 0

4.7 上板复验的实测结果 (PASS=25 FAIL=0 SKIP=6)

原文见 board/RESULT_audit.txt 。摘要：

```
① 地址别名    13/13 PASS
  5 个正例 (trng/key_vault/sym/mlkem/fan 的 VERSION) 全部读得到；
  8 个镜像地址 (+0x110、+0x100、+0x1100、+0xFF00、+0x8000、
  aperture 之外、槽号 6、槽号 7) 全部 DECERR。

② zeroize     4/4 PASS
  写 ZEROIZE 之后 STATUS.WIPING = 1；
  WIPING 持续 112.5 μs — 8192 拍 @75 MHz 理论 109.2 μs，差值是轮询开销；
  擦完 OUT_LEN = IN_PTR = 0，同一输入重跑 2432 字节逐字节相同。

④ TRNG        3/3 PASS
  起点 DROPS=0 BLOCKS=17892 — 这一行信息量最大：那 17892 个块是**没有人
  读随机字**的时候吸收的，而 DROPS 仍是 0。修复前这里是 65535 (饱和)。
  再取 4096 个随机字，BLOCKS +510，DROPS 仍为 0。

⑤ 非法参数    4/4 PASS
  mode=3 / pset=3 / 两者皆 3：PARAM_ERR 置位、BUSY 从未拉起、
  上一次的 DONE 与 OUT_LEN 已作废；换回合法参数 (768) 照常跑出 3584 字节。
```

算法侧回归同轮跑过 (board/RESULT_hwtest_after_audit.txt)：ML-KEM-512 的 ACVP 9 项、AES/SM4/SM3 的官方向量、密钥仓边界扫描 (48 读得到 / 80 被拒、零个密钥字)、金丝雀 DECERR 与对照、TRNG 健康检测，24/24 全过 —— 这一轮动了译码、擦除、TRNG 三处，算法结果一个字节没变。

上板暴露了两处仿真没覆盖到的东西

第一次上板的结果是 PASS=20 FAIL=6。六条红里有一条是板上程序自己写错了 (把 ML-KEM-768 的输出长度写成 2400，实际是 ek 1184 + dk 2400 = 3584)，其余五条指向两个真问题。两个都值得记，因为它们暴露的是仿真环境与真实使用方式的差距，不是 RTL 写错了字：

其一：没有人读随机字的时候，取样 FIFO 照样溢出。板上 DROPS 上电即饱和 到 65535。原因是输出 FIFO 满了之后调理器卡死在 S_SQUEEZE 等 word_ready，bit_ready 一直为 0，入口的取样 FIFO 256 拍就填满，此后每一个样本都被丢掉。仿真里从来没出现过，是因为测试台一直拉着 rd_en —— 而板上的常态恰恰相反：绝大多数时间没有人在读。加大 FIFO 没有用，卡住的时间没有上界。修法是让 调理器的出口永不阻塞：word_ready 恒为 1，输出 FIFO 满就丢弃那个调理后的字。丢一个已经调理好的 32 位字是完全无害的 (熵留在海绵状态里，duplex 模式一直 往前带)，而丢入口的样本直接破坏”被检测的序列 = 被使用的序列”那条前提。补的用例是”全程不读随机字”，在修复前的 RTL 上丢 454 个样本。

其二：START 被拒之后，上一次的 DONE 与 OUT_LEN 还留着。于是软件轮询到 DONE=1、读出 OUT_LEN=2432，拿着上一次的输出当成这一次的结果 —— 比不报错更糟，因为它看起来成功了。仿真里没出现，是因为每条用例都从复位开始，OUT_LEN 本来就是 0；板上是连着跑的。修法是一次 START 尝试就作废上一次的 结果，不管这次是否被接受。补的用例是”非法 START 发生在一次成功运行之后”。

这两条合起来说明一件事：仿真用例的默认姿势 (每次从复位开始、消费者一直在读) 本身就是一种未言明的假设。板上不按这个姿势来。

4.8 送检口径：全功能核 `SECURE_ONLY=1`，安全世界端到端跑完整 KAT

3.6 的边界证明成立，但它有一处够不着的地方：被门控保护的只有槽 4 那个金丝雀 空壳，不是密码核本身。功能核当时是 `SECURE_ONLY=0`（否则普通世界跑不了 KAT），于是”算法对”与”边界严”是在两个不同配置上分别证的。

这一版把这条缝合上：`trng_axi` / `sym_vault_top`（含 `key_vault` 与 `sym`）/ `mlkem_axi` 全部 `SECURE_ONLY=1`，普通世界一个寄存器都摸不到；整套 KAT 改由 安全世界驱动 —— 打过补丁的 BL31 提供一个受限的安全 MMIO 读写 SiP，普通世界的测试程序把每一笔核访问都经 SMC 转到 EL3 发出。

结果	
正向 ML-KEM 512/768/1024 KeyGen/Encaps/Decaps (ACVP 官方向量)	20/20 PASS (<code>board/RESULT_seckem3.txt</code>)
正向 AES-128/256、SM4、SM3、密钥仓边界扫描、金丝雀、TRNG 健康检测	24/24 PASS (<code>board/RESULT_sechwtest.txt</code>)
反向 同一批地址从普通世界直接读	6/6 PASS，四个功能核 + 金丝雀全部 DECERR (<code>board/RESULT_secneg.txt</code>)

一轮下来经 EL3 发出 294 万次读 + 522 万次写，被 EL3 白名单拒 161 次（全部来自边界扫描故意去读的窗口外地址）。

三个让这个证明站得住的对照

其一：同一个金丝雀地址，两种 transport 期望相反。

安全世界（EL3）读金丝雀 = `0x00010000`

普通世界直接读金丝雀 被总线拒

← 必须读到

← 必须读不到

只测”读不到”是不够的：一个把该地址写死堵掉的实现同样能通过。两条一起才说明 门控是按 AxPROT 判的，而不是按地址一律堵死。

其二：反证里带一个必须成功的对照。`hsm_secneg` 先读风扇观测口（槽 5，`SECURE_ONLY=0`）并要求成功——否则”四个核全读不到”可能只是 PL 没 配置、或 `/dev/mem` 这条路本身断了，那样上面五条什么都证明不了。实测读到 `0x00010000`。

其三：反证一笔写都不发。普通世界写一个被拒的地址，DECERR 是 posted 的，在 aarch64 上以 SError 回来，内核只能 panic —— 那不是一次失败的测试，是丢一块板子（这条代价是一次断电，见 `docs/fpga-进展.md`）。所以写侧的拒绝语义由仿真判，板上只读。

安全世界那个口子本身：怎么保证它不比它证明的东西更大

为证明边界而开的口，本身不能成为后门。四条自我约束（`boot/atf/patch_atf_secmmio.py`）：

- 白名单是 PL 内 `axi4lite_xbar` 译码判据的子集，绝不更宽。两处策略各说各话 正是 xbar 文件头里写的那个坑；
- 每槽的偏移上界引用各核自己的防火墙窗口（`ADDR_MASK`），不手抄寄存器表 —— 手抄的那一版就漂了：`sym_axi` 抄成 `0x30` 而它实际到 `0x60`，因为 `DOUT / DIGEST` 两块是用另一个表达式解码的，`grep localparam` 看不见。上板时当场被拒（失败方向是安全的，但根因是一处定义抄成了两处）；

- 金丝雀只读、风扇**整核排除**（它的 DRP 窗口被写坏会扰动 SYSMON 采样）；
- 边界与槽表全是**编译期常量**，判据函数无循环、无指针算术，14 行可一眼审完。顺带删掉两个不再需要的旧口子（`PROT_READ`、`PL_SECREAD`）——安全世界的口子只应该留正在用的那些。

内核模块（`board/kmod/secmmio.c`）**不是信任边界**：白名单全在 BL31 里，模块只当信使。这个分工是有意的——模块可以被 root 换掉，BL31 在 EL3 换不了。

发第一笔 SMC 之前的闸门放在**普通世界且不碰 PL 总线**：读 `/sys/class/fpga_manager/fpga0/state` 确认 `operating`。PL 不在就发 SMC 的话，那笔访问会在 EL3 上出错，而 BL31 里没有处理它的东西。

这一版与 3.6 的关系：3.6 证的是”门控这个机制生效”，用的是一个空壳；4.8 证的是”密码核本身在门控之后，且只有安全世界能驱动它们跑出正确结果”。后者把前者包含了。

5. 待办 / 受阻清单

真正剩下的受阻项只有 JTAG 那两件（5.1）。原先列在”需要安全世界主控”里的「把 XMPU/XPPU 配上」已经结案——不是完成，是**不适用**：UG1085 的地址表定论它结构上覆盖不到 PL 窗口（5.5）。安全世界主控本身该做的事（AxPROT 门控的双向闭合）已经完成，见 3.6。

5.1 需要 JTAG（线到位后即可做）

项	卡在哪里	到位后怎么做
风扇”开机就静”的持久化	两条不动 <code>BOOT.BIN</code> 的路都试过并失败：① <code>boot.scr</code> 里 <code>fpga loadb</code> 让 U-Boot 挂死；② Linux 早期 init 重配 PL 让启动挂住。 在这块板上”启动过程中重配 fabric”本身就不稳 。剩下唯一干净的做法是把风扇版 bitstream 直接放进 <code>BOOT.BIN</code> 的 PL 段让 FSBL 原生装（全程不重配）——但那要写 golden 槽， 没有 JTAG 写坏了没救	有 JTAG 后风险从”砖”降成”重刷”，直接换 <code>BOOT.BIN</code> 的 PL 段。打包路径已验证可用
BBRAM 安全启动	写 BBRAM 密钥必须走 JTAG	—

现状：风扇温控本身完全做通并验证，一条命令即可让板子安静（`sh /media/sd-mmcblk1p2/hsm/fanquiet-init.sh`），只是重启后需要重跑。

5.2 需要安全世界主控

项	现状
AxPROT 门控的完整闭合	✅ 已完成，见 3.6

项	现状
XMPU/XPPU 的”配上”	❌ 不适用，已结案（见 5.5）。不是”没做”也不是”做不动”：UG1085 v2.5 的地址表定论——XPPU 与 FPD_XMPU 结构上都覆盖不到 0x8000_0000 那段 PL 窗口。这一项从待办清单里移除，因为它从来就保护不了 PL 里的密码核
顺带查清的事实	这块板上 PS 侧保护单元完全未配置（XPPU CTRL=0、许可表 400 条全是复位默认值；XMPU_OCM 16 个区域一个都没使能），且 XPPU 没锁。与上面那条无关——即便配起来也够不到 PL

5.5 XMPU / XPPU：实测结论与定论

board/xmpu_probe.c 从普通世界读 XPPU、XMPU_DDR0..5、XMPU_FPD、XMPU_OCM 的全部控制与区域寄存器，共 149 次读：

```
=== ZynqMP 内存保护实配（普通世界视角） ===

[XPPU (LPD 从机的许可表) base 0xFF980000]
XPPU_CTRL @0xff980000 = **总线错误（硬件挡的）**
XPPU_ERR_STATUS1 @0xff980004 = **总线错误（硬件挡的）**
...
[反证：普通世界够不到的东西]
ok CSU_STATUS(0xFFCA0000) 读不到 — 普通世界确实够不到 CSU
ok CSU_MULTI_BOOT(0xFFCA0010) 读不到
ok eFUSE 控制器(0xFFCC0000) 读不到

[对照：同一条 /dev/mem 路径读 PL 是通的]
ok PL ML-KEM VERSION(0x80030000) — 路径本身是通的
```

146 次总线错误、0 次映射失败。这两种必须分清：前者说明 XPPU/XMPU 真的 在拦，后者只说明 /dev/mem 这条路被内核限制了、跟硬件隔离没关系。另加一条对照——同一条 /dev/mem 路径读 PL 地址成功，证明访问路径本身通。

结论：XMPU/XPPU 的配置寄存器自己就受保护，“把内存保护配上”这半件事 在普通世界不可能完成。

从安全世界看进去：这块板上的 PS 侧保护是完全未配置的

普通世界看不到不等于”配好了看不到”。给 BL31 加一个只读的保护单元读口（SiP 0x8200ff11，白名单只放行保护单元的寄存器窗口，见 boot/atf/patch_atf_protread.py），从 EL3 读出来的实配是：

寄存器	实测值	含义
XPPU_CTRL	0x00000000	XPPU 整个是关着的
XPPU_LOCK	0x00000000	没锁 —— 也就是说它可以被配
XPPU_APERPERM_000..399	400 条全是 0x00039ce7	复位默认值，许可表从未被编程
XMPU_OCM_CTRL	0x00000003	DEFRD\ DEFWR，默认放行读写
XMPU_OCM 的 16 个区域	全是 CONFIG = 0x08	ENABLE 位是 0，一个区域都没使能
白名单反证	读 DDR / BL31 自身地址均被 SiP 拒	白名单确实生效，不是”什么都读得到”

也就是说，FSBL 与 PMUFW 一条保护规则都没有配。文档此前写的”取决于 FSBL/PMUFW 配了什么”现在有了答案：什么都没配。

两个安全主控的能力不重叠 —— 这决定了只能走 BL31

同一批地址分别用两条路读，结果正好互补：

地址	EL3 的 SiP (APU 安全世界)	PM_MMIO_READ (PMU 代读)
XPPU (0xFF98_0000)	读到	拒 (2002 = XST_PM_NO_ACCESS)
XMPU_OCM (0xFFA7_0000)	读到	拒
XMPU_DDR0..5 / XMPU_FPD (0xFD00_0000 段)	页表未映射 (见下)	拒
CSU_MULTI_BOOT (0xFFCA_0010)	不在白名单	读到

对照是成立的：同一条 PM 通路读 CSU_MULTI_BOOT 成功，所以那些 2002 是 PMUFW 白名单的拒绝，不是 SMC 调用写错了。PMU 虽然是安全主控，却看不到任何一个保护单元——所以”走 BL31 的 SiP”不是绕远路，是唯一的路。

XMPU_DDR0..5 与 XMPU_FPD 至今没有读到，如实记：它们不在 BL31 的 页表里 (DEVICE0 是 0xFF00_0000+14MB、DEVICE1 是 GIC，都不覆盖 0xFD00_0000)，而加了那段映射的 BL31 在这块板上起不来——试了两次、断电两次，第二次把 MAX_XLAT_TABLES 从 7 放宽到 10 (BSS 确实涨了 12 KB，证明这次真的改到了) 仍然起不来，所以不是转换表数量的问题。为这两个次要 目标继续冒险不划算，停在这里。

定论：XPPU / XMPU 结构上覆盖不到 0x8000_0000 那段 PL 窗口

这一步最后没有变成”配了”，而是变成了一个更硬的结论。依据是 UG1085 的地址表 本身，不是推断。

证据一：XPPU 的 aperture 是穷举的，0x8000_0000 不在其中。UG1085 v2.5 第 404 页表 16–10 「XPPU Address Table」列全了四组 aperture：

大小	数量	aperture 号	保护的地址范围	配置寄存器
32B	128	256–383	0xFF99_0000 – 0xFF99_0FFF (IPI 消息缓冲)	0xFF98_1400 – 0xFF98_15FC
64 KB	256	000–255	0xFF00_0000 – 0xFFFF_FFFF	0xFF98_1000 – 0xFF98_13FC
1 MB	16	384–399	0xFE00_0000 – 0xFEFF_FFFF	0xFF98_1600 – 0xFF98_163C
512 MB	1	400	0xC000_0000 – 0xDFFF_FFFF (Quad-SPI)	0xFF98_1640

四组加起来覆盖 0xC000_0000–0xDFFF_FFFF、0xFE00_0000–0xFEFF_FFFF、0xFF00_0000–0xFFFF_FFFF。0x8000_0000 一个都不落，而这张表是完整的 ——不存在”第五组 aperture”。

证据二：XPPU 的职责定义本来就不含 PL。同一章第 400 页：

The XPPU is used to protect LPD peripheral and control registers (SLCR) along with message buffers (IPI) rather than memory (DDR, OCM). The XPPU is located in the LPD to protect the IOP ...

证据三：FPD_XMPU 也看不见这条路上的事务——而这里有一个很容易踩的陷阱。第 396 页表 16-6

「Peripherals Secured by FPD_XMPU」里确实有一行地址是 `0x8000_0000`，但那一行标的是 `PCIE_HIGH_1/2`，是 PCIe 的高位孔径，不是 PL 的 HPM 窗口。本设计不用 PCIe，那段窗口经 `M_AXI_HPM0_LPD` 路到 PL，而第 1092 页写得很明白：

LPD bus masters can use the M_AXI_HPM0_LPD interface to access the PL **without the FPD** ...

不经过 FPD，FPD_XMPU 自然看不见。照着表 16-6 那一行去配 FPD_XMPU，得到的正是“配了但没用”——而它和“配对了”长得一模一样，两者都不报错。另外，整个第 16 章（p392–420）里 `HPM` 一次都没有作为被保护的从口出现过。

所以：PS 侧没有任何保护单元覆盖 `HPM0_LPD` 这条路上的 PL 窗口。这不是“没找到”，是地址表穷举完了。

这是对架构的加固，不是缺口

结论反过来把这套设计的边界论证收紧了。此前文档里那句「PL 里密码核的边界 不靠 XMPU，靠 PL 内部的 AxPROT 门控防火墙」是一个设计判断；现在它有 TRM 的地址表背书，而且说法要更强一层：

在这条路由上，PL 内的 `axi4lite_firewall` 不是“多一层”，它是唯一一层。PS 侧没有任何东西能替它兜底——XPPU 够不着，FPD_XMPU 不在路径上。

由此推出一条直接的因果，值得单独记下来：

地址别名（4.6 ①）不是整洁性问题，它是这条唯一防线的命门。旧译码丢掉 `addr[15:8]` 与 aperture 以上的高位，于是每个寄存器都有大量镜像地址。如果 PS 侧还有一层按地址段做的保护，镜像至少还会被那一层挡一部分；而现在已经 确认没有那一层——镜像地址等于在唯一的防线上开了成千上万个等价入口，而且每一个都返回 OKAY、不留痕迹。

这也是为什么那一条被定为 P0、并且它的测试判据是“从机有没有被碰到”而不是“回了什么响应码”：在只有一层防线的系统里，“访问被正确地拒绝”和“访问根本到不了从机”是两个不同强度的命题，只有后者才是边界。修完之后实测：槽内 16384 个字地址里只有 `0x00` – `0xFC` 那 64 个可达，其余全部就地 DECERR（`test_xbar` 全槽扫描 + 板上 13/13）。

引用来源

本节所有结论出自 UG1085 《Zynq UltraScale+ Device Technical Reference Manual》v2.5，2025 年 3 月 21 日，AMD 官方（PDF 属性里 Author = Advanced Micro Devices, Inc.；1230 页）。页码均按该 PDF 的页码：第 16 章 System Protection Units（p392–420），其中 XPPU 职责 p400、表 16-10 XPPU Address Table p404、表 16-11 APERPERM 位定义 p405、表 16-6 Peripherals Secured by FPD_XMPU p395–396；`HPM0_LPD` 绕过 FPD 的表述在 p1092。

5.3 永不做（已定死）

项	原因
eFUSE	不可逆。只有一块板，一个 bit 都不烧
PUF 黑钥	依赖 eFUSE 认证启动，同上

5.4 本次范围外

项	说明
DPA / 电磁侧信道	原型不做，也不假装做了。4.2 的常数时间抽测只覆盖计时这一层
SP 800–90B 重启测试	本次采集是有间隙的抽样，结构不符合那一项的要求
ML–DSA 整核	算子已实现（13 个模块）并过对拍，未整核化、未上板
ppc_accel_axi 的独立 KAT	它的批量数据走 AXI4–Stream，要 DMA 才能从 PS 驱动。其中的 ntt_core 与 sha3_core 并未因此失去验证——ML–KEM 每次运算都跑满 SHA–3 海绵和多次 NTT，ACVP 向量对上即等价于这两个核在真硬件上是对的。但这是间接覆盖，不是独立 KAT

6. 复现方法

```
# 仿真回归 (197 项, Mac/Linux 本地即可)
./tools/rtl_sim.sh

# 静态检查
./tools/rtl_lint.sh

# SP 800–90B 估计器自检 (复现规范算例)
python3 tools/sp800_90b.py --selftest

# 出 bitstream (构建机, Vivado 2020.1, 约 35 分钟)
vivado -mode batch -source hardware/syn/impl_bitstream.tcl
# 产物 hardware/syn/impl/zu3eg_hsm.bit
# 表征版 (低风扇阈值 + TRNG 原始抽头, **不是产品形态**):
PQC_CHARACTERIZE=1 vivado -mode batch -source hardware/syn/impl_bitstream.tcl
# 产物 hardware/syn/impl/zu3eg_hsm_char.bit

# 上板 (所有碰 PL 的动作都必须经过 harness, 理由见 3.5)
sh /media/sd-mmcb1k1p2/hsm/plharness.sh <payload.sh>
```

关键文件

文件	作用
hardware/rtl/board/zu3eg_hsm_top.v	板级顶层，所有集成决策都在它的注释里
hardware/syn/impl_bitstream.tcl	从 RTL 到 .bit 的完整流程 + 四条断言

文件	作用
<code>board/plharness.sh</code>	碰 PL 的唯一入口（脱离终端、收尾必恢复网络、看门狗）
<code>board/hsm_hwtest.c</code>	24 项自测（ML-KEM-512 + 对称 + 边界 + AxPROT + TRNG）
<code>board/hsm_kem3.c</code>	ML-KEM 三参数集 KAT + 性能 + 常数时间
<code>board/trngraw.c</code>	导出调理前原始噪声比特
<code>board/xmpu_probe.c</code>	XMPU/XPPU 实配探测 + 反证
<code>boot/atf/patch_atf_secread.py</code>	给 BL31 加 EL3 读 PL 的 SiP
<code>boot/atf/patch_atf_plmap.py</code>	给 BL31 页表补 PL 窗口映射（缺了会卡死在 EL3）
<code>board/kmod/pmsec.c</code>	发 PM SMC / 自定义 SiP 的内核模块
<code>tools/sp800_90b.py</code>	SP 800-90B 十项估计器（含规范算例自检）
<code>docs/fpga-进展.md</code>	逐阶段的详细工程日志（S1-S7、P6、P7）

7. 一句话总结

一台完整的后量子密码机原型跑在 ZU3EG 的 FPGA 逻辑里，占片子不到一半：ML-KEM 三套参数集对 NIST 官方向量逐字节一致，AES/SM4/SM3 对标准向量逐字节一致，密钥”能用但读不出来”有反证，访问门控在真硅上被证明生效，真随机数源有实测最小熵 0.871234 比特/样本（并据此纠正了两个原本站不住的 健康检测阈值——其中 APT 那道检测原来根本不会触发），风扇温控闭环验证。

访问门控已完整闭合：同一个地址，安全世界（EL3，`AxPROT[1]=0`）读到 `0x00010000`，普通世界（EL1-NS，`AxPROT[1]=1`）被总线拒绝——而同一个普通世界能读到 `SECURE_ONLY=0` 的核，所以差别确实来自门控本身（见 3.6）。

剩下的边界项只有两类：需要 JTAG 的（开机即静的持久化、BBRAM 安全启动），和 XMPU/XPPU 的实际配置（要安全世界主控去配，本次只做了门控证明所需的 最小安全负载）。